Changelog: v20 → v21 → v22

Survival search, free steps, routing, landing safety, rushes and the king's plan;
v22: the judge's points budget

Summary

v21 was built in five rounds.

Part I, the survival search. A depth-first *survival search* (`survival.hpp`) asks, for each candidate move, whether our dragon can keep moving for its own length plus two more turns, playing out the engine's movement rules exactly. Its result feeds the safety filter (`critical_ok`) and the position score. Two supporting changes came out of testing: traps are priced by whether a split-escape is actually possible, and workers of length ≥ 8 may use the reserved escape slot at the unit cap.

Part II, the free-steps rule and routing.

- The game update's sprint rule ($\lceil L/4 \rceil$ free steps a turn) is now modelled everywhere the bot reasons about movement: its own moves, enemy reach and the simulator.
- The bot uses the rule itself: besides single steps it weighs *free moves* of several steps.
- Pearl routing (`route.hpp`) is rebuilt around four ideas:
  - time-aware paths: bodies are passable once they have moved on;
  - arrival measured in turns rather than steps;
  - who gets to each pearl first, by real paths from every visible head;
  - a survival search from where each route ends, so that a route that would box us in, above all the king, is never chosen.
- Testing showed free moves could carry the king to the edge of its vision, next to enemies it could not see, so valuable dragons now price that exposure.

Part III, landing safety and teammate memory. Free moves never leave the 7×7 window, but they can *end* at its edge. Long snakes then got boxed in, mostly by allies they could not see, 5–10× more often than after a single step. The changes:

- bodies are remembered for as long as they will really be there;
- allies' moves are modelled from what we see and from their status packets;
- long snakes pay a price for landing next to tiles they cannot see;
- the king no longer outruns its feeders in the late game.

Part IV, going back for remembered pearls. A long king that left pearls behind, such as pearls 30 steps away around its own body, never went back: routes only looked 20 steps ahead, which is one turn for a king of 80. The changes:

- routes now reach a few turns of free steps;
- a *rush* follows the route for all our free steps, also over ground we cannot see now but have seen, priced by how likely it is that something has moved there since;
- bodies running out of sight are predicted where they continue.

This round also found and fixed a bug that predates v21. Dragon ids pass 4096 in long games and our 12-bit status packets wrapped them, so feeders lost track of the king and teams were wiped out.

Part V, the king's own moves. Three changes for the king and growers:

- the survival search now proves a way on for their whole length, not 20 moves;
- a dragon that does not know all of its body treats unseen tiles next to where it ends as possibly its own body;
- each of the king's best moves is weighed by a search over its own multi-step moves for the next two turns.

Measuring showed that kings rarely die to their own body alone. They get boxed in by other dragons and then split-escape about once in ten turns, often in chains (see §27).

The queens/win-condition update is **not** implemented.

v22 (Part VI), the judge's points budget. The judge meters every dragon's turn in CPU points (100M a turn); a turn over it gets no reply, and the dragon dies with "no valid action". Measured with the judge's own pricing, v21's long dragons went over on hundreds of turns a game (kings averaged 164M a turn, up to 1.2 billion). v22 reads its own spending off the judge's clock, which counts points, and stops each search at its phase's deadline. The same searches were also made cheaper: a survival-search position by 2.5×, the king's plan options by over 10×. No measured turn of v22 goes over 87M, and v22 keeps v21's lookahead wherever it fits: the budget changes the chosen move on 0% of turns below length 100 and about 8% at 150+.

File	Change
`survival.hpp`	**New**: the survival DFS (Part I); bodies-only timings for routing, free-step head reach, `survival_from` for route ends (Part II); remembered bodies' lifetimes, allies' known lengths and status packets (Part III); deep searches, unknown-body caution, the king's plan (Part V)
`route.hpp`	**Rewritten** (Part II): time-aware searches, ETAs from every visible head, turn-based route values, BFS-guided replay with a route-end survival check, free moves; no exploring free moves for the late-game king (Part III); route range in turns, rushes (Part IV); deep route-end checks for the king (Part V)
`main.cpp`	Movement-rule helpers; config; `Candidate` fields; turn order; safety filters; escape slot; `simulate_path`, threat maps, reach threats, sprints and the ally contest under the new rule; enemy-density tracking; body memory (`busy_until`, `remember_bodies`), `known_length`, ally density (Part III); fill rate and occupancy, remembered tiles in `simulate_path`, hidden segments, 12-bit ids (Part IV); deep-search and plan constants, `Candidate::plan_value`, plan hook (Part V)
`scoring.hpp`	`trap_cost`; new `position_value` branches; goal progress for moves of k steps; route progress; fight reach; `unseen_risk` (Part III: a box-in part for every dragon of 10+)
`explore.hpp`	Pearl sprints: survival search, new rule, contest by turns; `disperse_score` per step
`endgame.hpp`	Death-yield comment; "cheap way out" by paid steps; `feeding_score` per step; 12-bit ids (Part IV)
`coordination.hpp`	12-bit ids (Part IV)
`helper.hpp`	Replaced with the upgraded-protocol release (still `PROTOCOL_MAJOR = 3`). Against ours, only the `make_moves` comment differs: it now states the free-steps rule (the first $\lceil L/4 \rceil$ steps free, at least 2 segments kept), which the bot and `sim/engine.py` already follow. The compiled bot is byte-identical.
`sim/engine.py`	The free-steps rule (`FREE_MOVES` switch)
`sim/` probes	`probe_aggr.py` (metrics), new `probe_trapped.py`, `probe_bigdeath.py`, `probe_turn.py`, `scen_route.py`, `probe_freemove.py`, `probe_hitbody.py`, `ab_trap.py`, `probe_fill.py`; `scen_route.py` gains `loopback` and `ownpocket`; `probe_selfdeath.py`, `probe_kingtrap.py`, `probe_ksplit.py`, `probe_kroom.py`, `probe_kingself.py`, `probe_bodyk.py`

Contents

I The survival search **6**

1 Movement rules the search is built on **6**

2 New file: `survival.hpp` **6**

2.1 What it computes . . . 6
2.2 State (new members) . . . 7
2.3 `prepare_survival()`: once per turn . . . 7
2.4 `survival(pm, heading)`: per candidate . . . 7
2.5 `surv_dfs`: the search . . . 8

3 Part I changes outside `survival.hpp` 8
3.1 Configuration (`main.cpp`) . . . 8
3.2 `Candidate` fields . . . 8
3.3 Include . . . 9
3.4 Escape slot policy (`escape_slot_available`) . . . 9
3.5 Safety filters . . . 9
3.6 Scoring (`scoring.hpp`) . . . 9
3.7 Debug log and comments . . . 10

4 How the search got here 10

5 Part I results (old movement rule) 10

II The free-steps rule, free moves and routing 10

6 The free-steps rule 10
6.1 The rule as modelled . . . 10
6.2 Where the bot now uses it . . . 11

7 Free moves 11
7.1 Generation (`add_free_moves`, `route.hpp`) . . . 11
7.2 Evaluation (`evaluate_path`) . . . 12
7.3 Goal progress for k steps (`scoring.hpp`) . . . 12

8 Routing (`route.hpp`) 12
8.1 Time-aware paths . . . 12
8.2 Arrival in turns . . . 13
8.3 Who gets there first . . . 13
8.4 Replay and the route-end check . . . 13
8.5 Route progress . . . 14
8.6 Turn order . . . 14

9 Valuable dragons: what they cannot see 14

10 New and changed constants (Part II) 14

11 Part II testing (new movement rule, against v21 Part I) 15
11.1 Routing scenarios (`sim/scen_route.py`) . . . 15
11.2 Games against v21 Part I (both under the new rule) . . . 15
11.3 Metrics (seed 20, 56×53, 200 rounds) . . . 15
11.4 Other checks . . . 16
11.5 How Part II got here . . . 16

III Landing safety and teammate memory 16

12 What was actually happening 16

13 Body memory 17

14 Where allies can be before our next move 17

15 Landing next to tiles we cannot see 17

16 The late-game king keeps to its feeders' pace 18

17 Part III results 18

IV Going back for remembered pearls 18

18 What was wrong 18

19 Route range in turns 19

20 Rush 19

21 Bodies running out of sight 20

22 The 12-bit id bug (predates v21) 20

23 Part IV results 20

V The king's own moves 20

24 Deep self-safety 21

25 A body we do not know 21

26 The king's plan 21

27 How kings actually get trapped 22

28 Part V results 22

29 Known limitations and not yet done 22

VI v21 → v22: the judge's points budget 23

30 What the judge measures 23

31 Measuring it 24

32 v21's turns 24

33 The turn's clock and deadlines 24

34 Cheaper searches 25

35 Part VI results 26

36 Checked in the sandbox, and a 95M peak 27

37 Tools added 28

38 v22: known limitations and next steps 28

VII v22 (continued): the queen 28

39 The rule 28

40 What the queen does (queen.hpp) **29**

41 A bug in the native turn clock **29**

42 The simulator's pearls are far too rich **30**

43 Queen results (simulator, so indicative only) **30**

44 The queen grows the swarm until round 75 **30**

45 When the queen is certainly dead, she saves the rest of her length **31**

VIII v22 (continued): the centre rush on small maps **31**

46 Which maps **31**

47 What it does **32**

48 Simulator check (random maps, pearls stretched 30×: indicative only) **32**

Part I
The survival search

1 Movement rules the search is built on

These were confirmed before writing the search and are what `surv_dfs` simulates:

1. **Self-collision includes the tail**, and is checked *before* the tail advances. Within a multi-step move, each step is checked against the body as the previous step left it.
2. **Dragons act one at a time**, and each sees the actual board at its turn.
3. **Sprints** check collisions at every step and remove one extra tail segment per paid step (Part II: the first $\lceil L/4 \rceil$ steps of a turn are free).
4. **Pay before eat**: the validity check for a step happens before that step's pearl is eaten, so a pearl does not save a dragon on its death step.
5. **Death yield**: a dragon of length L always drops $\lceil L/2 \rceil$ pearls.
6. **No simultaneous entry**: two dragons never move into a tile at the same time.

2 New file: `survival.hpp`

Included inside `ReplayDerivedBot`, before `scoring.hpp`.

2.1 What it computes

```
struct SurvivalResult {
    int depth = -1; // most moves survived on any path (-1: not searched)
    int target = 0;
    bool complete = false; // every path searched (budget did not cut it short)
    bool ok() const { return depth >= 0 && depth >= target; }
    bool dead() const { return depth >= 0 && complete && depth * 4 < target * 3; }
};
```

- **Target depth**: $\min(L + \texttt{DFS_EXTRA_DEPTH},\ \texttt{DFS_MAX_DEPTH}) = \min(L + 2, 20)$, where L is our length after the candidate move. Surviving L moves means our whole current body has moved off its tiles; the +2 is margin.
- **ok**: some path reached the target.
- **dead**: the search finished in full and the best path is short of 3/4 of the target. This is a trap.
- Anything else (including a budget cut) is "unknown", and the old v20 heuristics decide.
- Our own moves are searched *one step a turn*. Free steps are optional, so we can always go that slowly, and that pace gives other bodies the most turns to move on.

2.2 State (new members)

- `other_free_at_`: per tile, how many moves the other dragons must have made before the tile is free (`INF`: never).
- `body_free_at_` (Part II): the same for *bodies only*, meaning other dragons' segments and remembered bodies. Our own segments and the head-reach margin are left out. Routing and the ETA searches use it.
- `surv_occ_`: our body during the search (occupancy counts).
- `surv_eaten_`: pearls eaten along the current path, so the same pearl is not eaten twice.
- `surv_nodes_`, `surv_turn_nodes_`: per-candidate and per-turn node counters.
- Part II: `surv_route_nodes_` (the route checks' own per-turn counter), plus `surv_counter_`, `surv_cap_` and `surv_offset_`, which select the counter, cap and time offset for the current search.

2.3 `prepare_survival()`: once per turn

Called from `play_turn`. Part II moved it before the objective map and routing (see §8.6). It resets both node counters and fills `other_free_at_` and `body_free_at_`:

1. A tile with a **remembered** (out-of-sight) body blocks for $\texttt{DFS_REMEMBERED_STEPS} = 3$ moves (both maps).
2. **Our own unplaceable segments** (`own_idx_ == -2`, never seen since birth) free after $\max(1,\ L - \texttt{known_body_})$ moves, since they are in the unknown tail (`other_free_at_` only).
3. **Every visible part of another dragon** is a wall (`INF`) by default (both maps).
4. For each **visible head**, the body is traced (`trace_body`). Segment i frees after $\max(\texttt{DFS_OTHER_MIN_WAIT},\ est - i + 1)$ moves, where *est* is that dragon's estimated length (both maps). Each dragon moves once between two of ours; the +1 covers it eating (its tail then stays); and nothing frees before our second move. Segments whose head we cannot see stay walls.
5. **Head reach**: tiles a visible head could be on before our next move get a value of at least 2, so they are not counted on for our first step. In Part I these were its non-reverse next tiles. In Part II they are every tile within `free_moves(est)` steps (`tiles_within`), since its body follows it there. This applies to `other_free_at_` only; where heads go after that is left to the fight search.

2.4 `survival(pm, heading)`: per candidate

- Builds our body after the move, head first: `pm.path` reversed, then `own_trail_`, truncated to `pm.length`. Any remaining length is an `unknown` count of tail segments we cannot place.

- Part II: the search itself is now `survival_from(body, unknown, heading, length, offset, route)`, which `survival` calls with offset 0 and the moves' budget.
- If the per-turn budget is already spent, it returns "not searched" (`depth = -1`).

2.5 `surv_dfs`: the search

- Stops with success at the target. It stops with `cut = true` when either budget runs out: `DFS_NODE_BUDGET` = 4000 per search, and `DFS_TURN_BUDGET` = 40000 per turn for moves. Route checks have their own `ROUTE_DFS_TURN_BUDGET` = 24000.
- **Children** are the (up to three) non-reverse directions whose destination is passable and known. The tile must also hold none of our body (tail included, per rule 1) and have `other_free_at_` $\leq$ *step* + `surv_offset_`. Kelp blocks, the map wraps, and known portals are followed. Unknown portals are not taken, since we cannot see where they lead.
- **Ordering**: children are sorted by *openness*, the count of free neighbours at the next step, most open first. In open water the first path reaches the target, so the search is cheap; the budget only bites in tight spots. This is a manual insertion sort over at most three elements (see §4).
- **Moving**: the head is pushed. If the tile has a pearl seen this round that is not already eaten on this path, we grow and the tail stays. Otherwise an unknown tail segment is consumed if any remain, else the real tail tile is popped.
- **Backtracking** undoes every change: the head push, the tail pop, the unknown count and the eaten marks.
- Records the deepest depth reached in `best`.

3 Part I changes outside `survival.hpp`

3.1 Configuration (`main.cpp`)

```
#ifndef LONG_WORKER_ESCAPE
#define LONG_WORKER_ESCAPE 8
#endif
constexpr int LONG_WORKER_ESCAPE_LENGTH = LONG_WORKER_ESCAPE;

// v21: survival search
constexpr int DFS_EXTRA_DEPTH = 2; // target = length + this ...
constexpr int DFS_MAX_DEPTH = 20; // ... capped here
constexpr int DFS_NODE_BUDGET = 4000; // per search
constexpr int DFS_TURN_BUDGET = 40000; // per turn (moves)
constexpr int DFS_REMEMBERED_STEPS = 3; // remembered body blocks this long
constexpr int DFS_OTHER_MIN_WAIT = 2; // no other segment frees before this
constexpr double P_DFS_SHORT = 0.9; // trap probability per unit shortfall
```

`LONG_WORKER_ESCAPE` sits after `ESCAPE_UNIT_RESERVE = 2`. Its comment records why it exists: boxed-in workers at the unit cap used to die whole, about 140 deaths of dragons of length 10+ per 200 rounds on a big map. Compiling with `-DLONG_WORKER_ESCAPE=1000` turns the feature off.

3.2 `Candidate` fields

```
int dfs_depth = -1, dfs_target = 0; bool dfs_complete = false;
bool dfs_ok() const { return dfs_depth >= 0 && dfs_depth >= dfs_target; }
bool dfs_dead() const { return dfs_depth >= 0 && dfs_complete
                           && dfs_depth * 4 < dfs_target * 3; }
```

3.3 Include

`##include "survival.hpp"` is added before `scoring.hpp`.

3.4 Escape slot policy (`escape_slot_available`)

Previously a worker always had to leave `ESCAPE_UNIT_RESERVE` = 2 unit slots free before it could split-escape. Now:

```
reserve = king ? 0
       : (valuable_now_ || length >= LONG_WORKER_ESCAPE_LENGTH) ? 1
       : ESCAPE_UNIT_RESERVE;
```

A worker of length ≥ 8 may use the second-to-last slot to split out of a trap, like a valuable dragon. Losing 8+ segments whole costs far more than one extra unit near the cap.

3.5 Safety filters

In `evaluate_single`, after the room facts are gathered, `survival(pm, direction)` runs and is stored in the candidate. The room part of `critical_ok` is now:

```
room_ok = dfs_ok()
      || (!dfs_dead() && exits > 0
          && space >= pm.length + unknown_body() + 2 && !trap);
```

- A move the search proves survivable is safe even when the flood-fill heuristics call it tight.
- A move the search proves a trap is unsafe even when they look fine.
- Unknown falls back to v20.
- The other conditions are unchanged: known destination, no unknown portal, no head trade, no occupied exit, and for valuable dragons no hard threats or blind portal.

The same rule, run from each sprint's end, decides `critical_ok` for escape sprints (with `hard_threats == 0`) and pearl sprints (with `!valuable || hard == 0`).

3.6 Scoring (`scoring.hpp`)

`trap_cost`. In v20 a trap was priced as if a split-escape were always available (2 segments plus a unit). At the unit cap it is not, and the dragon dies whole, so traps were underpriced by about 3×. Now:

```
bool const can_escape = L >= 4 && escape_slot_available()
                     && controller_.can_split(L - 2);
return can_escape ? seg(2.0 + Config::UNIT_VALUE_OURS) : death_cost();
```

`position_value`: new branches after the pocket-plan case.

- `dfs_ok()`: the room is proven. Only the `P_ONE_EXIT` risk is charged when `exits == 1`; the space and dead-end risks are skipped.
- Searched in full but short: `v -= risk(P_DFS_SHORT * shortfall, trap_cost())`, where $shortfall = (target - depth)/target$.
- Not searched, or cut short by the budget: the v20 exits/space/dead-end risks, unchanged.

3.7 Debug log and comments

- The `CAND` debug line ends with `dfs <depth> <target> C|-`, where `C` means the search was complete.
- `endgame.hpp`: "Full yield" was misleading, so the comment now reads: *Every death leaves $\lceil L/2 \rceil$ pearls on the body's tiles; dying on our own neck here (rather than in a trade) puts them where the king will eat them.*

4 How the search got here

1. **First version (optimistic).** Other dragons' tails freed at step 1 and other heads were ignored. Since `dfs_ok()` drops the `exits == 0` risk, dragons stepped onto tiles with no free neighbour.
2. **Fix: cautious timing.** Added `DFS_OTHER_MIN_WAIT = 2` and the $est - i + 1$ eating margin, and blocked other heads' next tiles for our first step.
3. **Fix: trap pricing.** Nearly all self-deaths were `TRAPPED_SELF` (no legal move) at ≥ 62 units, where workers cannot split-escape. `trap_cost` now checks `escape_slot_available() && can_split`.
4. **Escape slot.** Even correctly priced traps still happen at the cap, so long workers were allowed the reserve slot.
5. **Compiler warning.** `std::sort` on `std::array<Child,3>` triggered a GCC `-Warray-bounds` false positive. It was replaced with a manual insertion sort.

5 Part I results (old movement rule)

No-legal-move deaths in 200 rounds on seed 20 against the baseline:

Version	Trapped deaths	of which length 10+
v20	318	157
v21, cautious DFS	297	–
v21, + trap pricing	270	–
v21, + escape slot	238	92

- Walls around trapped heads were mostly allies (ally body 463–499, ally head 86–111), then own body (∼450), kelp (∼120) and enemies (∼25).
- Full games against v20: 2–2 without the escape slot; **4–2** with it, with accidental self-deaths about 27% lower on big maps.
- The scenario tests passed and the coil stress test failed exactly as v20 did; the maximum turn time was 5–13 ms.

Part II
The free-steps rule, free moves and routing

6 The free-steps rule

6.1 The rule as modelled

In one turn a dragon of length L may take $\lceil L/4 \rceil$ steps for free. Every step after those costs a segment (its tail loses one more) and is allowed only while the dragon is longer than 2. That gives 1 free step at lengths 1–4 (the old rule), 2 at 5–8, 3 at 9–12 and 5 at 20.

Assumption: the allowance is per turn, and L is the length at the *start* of the turn.

```
constexpr int free_moves(int length) { return std::max(1, (length + 3) / 4); }
constexpr int paid_steps(int length, int steps) { return std::max(0, steps - free_moves(
    length)); }
// The most steps it can take in one turn (eating nothing on the way).
constexpr int max_turn_steps(int length) { return free_moves(length) + std::max(0,
    length - 2); }
// Turns to take `steps` steps at full free speed.
constexpr int turns_for(int steps, int length); // ceil(steps / free_moves(length))
```

6.2 Where the bot now uses it

Place	Change
`simulate_path`	The pay check applies only to paid steps (`s >= free && length_now <= 2`). Length before step s is $L + eaten - \max(0, s - free)$; after the move, $L + eaten -$ `paid_steps`.
`play_turn`	The projected length sent in the status packet subtracts `paid_steps`.
Reach threats (valuable)	Hard reach is `max_turn_steps(lower_length)` (was `lower_length - 1`). Soft reach for a truncated body is `max_turn_steps` of the larger of the typical enemy length and that dragon's own estimate (what we see +2), still capped by `HIDDEN_BODY_REACH`.
`build_threat_maps`	An enemy head threatens every tile within its free steps (`tiles_within`); an ally's next step is still what crowds us. For a reach of 1 this is the same set as before.
Fight search	An enemy's sprint into our head reaches `clamp(max_turn_steps(len), 1, max(FIGHT_SPRINT_REACH, free_moves(len)))`.
Survival search	Head reach covers its free steps (Part I, `prepare_survival`).
Tactical (attack) sprints	Up to `min(max_turn_steps(L), MAX_ATTACK_STEPS)` steps; the tie-break cost is per *paid* step.
Escape sprints	Up to `min(max_turn_steps(L) - 1, MAX_ESCAPE_STEPS)` steps; the score is `(eaten - paid) * SEG`.
Pearl sprints	Only for pearls beyond our free steps (closer ones are free moves). Net is eaten minus paid. "Contested" means an enemy needs no more turns than we would at free speed (by ETA, §8.3).
Split escape	A "cheap way out" is any safe move paying at most 2 segments (was: at most 3 steps).
Simulator	`engine.py`: `FREE_MOVES = True` applies the rule (pay check and extra tail segment only past the free steps).

7 Free moves

7.1 Generation (`add_free_moves`, `route.hpp`)

Each turn, after the single steps and the tactical and pearl sprints (and before escape sprints), we weigh moves of 2 up to $\min($`free_moves`$(L),$ `FREE_MOVE_MAX_STEPS` $= 6)$ steps:

- A depth-first walk over step sequences (never reversing). Every step is through a tile we can see now, with no portal. Each prefix is checked with `simulate_path`, which uses the exact engine rules.
- One option is kept per (end tile, last direction), the best by a quick look (`quick_value`): food on the way, exits at the end, and goal progress (along the route by the pearls' own maps).
- The best `FREE_MOVE_EVALS` $= 6$ are evaluated in full by `evaluate_path`.

7.2 Evaluation (`evaluate_path`)

It gathers the same facts as `evaluate_single`: threats at the end, exits, time-aware space and region, known space (valuable), and the survival search. The move is priced as in `move_value`:

- goal progress for k steps (half strength in a fight);
- every pearl eaten on the way (`path_food`), minus paid steps (none);
- `position_value` at the end and, in a fight, `fight_value` from there;
- visits to the end tile, teammate spacing (unless it eats), and `BLOCKED_SPAWN` for every path tile about to spawn;
- the enemy-distance pull.

`critical_ok` and `safe` follow the single-step rules. The new `Candidate::free_move` flag makes the chosen move log as `FREE_MOVE`.

7.3 Goal progress for k steps (`scoring.hpp`)

`goal_progress` now takes the `PostMove` instead of a `Step`, so the same code prices one step or several. With $k = 1$ every term is as before.

Goal	Progress
EAT	Steps still to go along the route, before minus after, clamped to $\pm k$ (§8.5); `ROUTE_TIE` if the move follows the checked route path.
HUNT	Grid distance gained to the target (unclamped, as before).
FEED	`feeding_score(dest, k)`: the gain clamped to $\pm(k+1)$ (was ± 2).
EXPLORE	Objective-map gain capped at $k+1$ (was 2); `disperse_score(dest, k)` clamped to $\pm k$; rendezvous unchanged.

8 Routing (`route.hpp`)

8.1 Time-aware paths

`timed_bfs(pm, limit, target, out)` gives the fewest steps for our head to reach a tile from where a move `pm` leaves us (or from where we are now). Every body is placed where it will be by then:

- Our own segment j (after the move) is passed once the tail has moved past it: at least $\mathit{length} - j + 1$ more steps.
- Another dragon's segment is passed once that dragon has made `body_free_at_` moves, one a turn. Our step is assumed to come in turn `turns_for`(*step*), at full free speed, plus the current turn if a move is already being made. Going that fast is pessimistic about other bodies.
- Kelp, tiles we know nothing of, and marked traps block.

It is used in three places:

- `route_head_dist_`: the map from our head, out to 30 steps (`prepare_etas`).
- The pearls' own maps (legs between pearls): a body tile is passable if our time-aware map reaches it, since it will have moved on by then. The first leg of every route uses `route_head_dist_` directly.
- Route progress for candidate moves (§8.5).

8.2 Arrival in turns

A pearl reached at step t is reached in `turns_for`(t, L) turns. The discount (0.92 for keepers, 0.85 for workers) is now *per turn*; for $L \leq 4$ this is the old per-step rule. Orders that arrive in the same turns are told apart by a 10^{-4}-per-step tie-break. Candidate pearls are filtered by time-aware steps within the route radius (20 for keepers, 14 for workers).

8.3 Who gets there first

`prepare_etas()` runs once per turn, before the objective map. It computes our time-aware map and one search per visible head (`head_eta_bfs`, 30 steps):

- That head's own segments pass once it has moved on.
- Our segments pass once we have made that many moves (we move before each of its turns).
- Others pass per `body_free_at_`.

Enemies are merged into `enemy_eta_turn_` and `enemy_eta_steps_`; allies are kept per head (`ally_etas_`).

The results are used in four places:

- **Route values**: a pearl an enemy reaches in fewer turns than our arrival on that route is worth `ROUTE_CONTEST_LOST` = 0.4 of itself. Before, any pearl closer to an enemy by *grid* distance than to us by path was halved. We win ties: every other dragon's t-th turn comes after ours.
- **Keepers**: the fight-risk factor (0.4) now needs an enemy within `ROUTE_ENEMY_NEAR` = 4 *path* steps of the pearl.
- `pearl_taken_by_ally`: turns at each dragon's own free speed, by time-aware paths for us and visible allies. Allies out of sight still use grid distance, now in turns. Ties still go to the lower id, a rule both dragons follow.
- **Pearl sprints**: contested by turns (see §6).

8.4 Replay and the route-end check

`replay_route` walks a route as the engine would move us, with our body trailing and growing:

- Each leg follows a shortest time-aware path from wherever the replay has got to (`replay_leg`): a BFS that knows the replayed body's segment indices, with the most open first step preferred.
- If the growing body gets in the way, the leg is searched again, at most 3 times per leg.
- A replay may take at most `ROUTE_REPLAY_SLACK` = 8 steps more than the horizon.

From where the route ends, the survival search runs (`survival_from`) on the replayed body:

- The time offset is `turns_for`$(t) - 1$ other-dragon moves; the pearls the route ate are marked eaten.
- It uses its own per-turn budget, so routing never starves the move evaluation.

The route is accepted only if that search succeeds:

- **King and growers**: `ok()`, or cut short by the budget after reaching at least 3/4 of the target.
- **Workers**: not `dead()`.
- **Budget already spent**: the old check (at least one free neighbour) decides.

The fallback (the nearest pearl on its own) goes through the same check. The replayed tiles are kept in `route_path_`, its first tile in `route_next_cell_`, and the order in `route_seq_`.

8.5 Route progress

`route_remaining(pm, limit)` counts the steps still to go after a move:

- time-aware steps (`timed_bfs`) from where the move leaves us to the first route pearl it has not eaten;
- plus the legs between the rest, skipping pearls eaten on the way.

`route_remaining_now_` is the same from here. EAT progress is the difference, clamped to $\pm k$; a move that eats the first pearl and carries on toward the second is credited for both. `follows_route(pm)` gives `ROUTE_TIE` when the move's tiles are a prefix of `route_path_`. The old `route_dist()` is removed.

8.6 Turn order

```
update_explore_target();
prepare_survival(); // v21: when other dragons' bodies move on (survival.hpp)
prepare_etas(); // v21: who reaches which tile first (route.hpp)
build_objective_map();
plan_route();
choose_goal();
...
add_tactical_sprints(candidates);
add_pearl_sprints(candidates);
add_free_moves(candidates); // v21
add_escape_sprints(candidates);
```

The debug `ROUTE` line now shows `rest <route_remaining_now_>` and `end_dfs <depth>`.

9 Valuable dragons: what they cannot see

A test game showed the danger. A king of length 111 made a 6-step free move to the edge of its window and landed next to an enemy of length 6 it could not see; the enemy rammed it with a free 2-step move. A single step never ends closer than 2 tiles to the window's edge; a free move can end on it.

`unseen_risk(pm)` (`position_value`, valuable dragons only) prices this.

- Count the tiles within `UNSEEN_RADIUS` = 3 steps of where we end that we cannot see now. Every single step has exactly one; an end at the window's edge has up to a dozen.
- Each tile is a chance of `P_UNSEEN_ATTACK` = 0.5 times the enemy heads we see per tile. That density is a running average (`enemy_density_x1e4_`, updated in `observe_heads`).
- The total is capped at `P_UNSEEN_CAP` = 0.3, times `death_cost()`.
- A blind portal exit is not counted here, since portals are priced separately.

A second king death (length 85) came from a body running out of sight. It was assumed only as long as a typical enemy, reaching 3 steps, but it was 8 long and paid 3 segments to reach 5. The soft reach now also uses that dragon's own estimated length (see §6).

10 New and changed constants (Part II)

```
constexpr int FREE_MOVE_MAX_STEPS = 6, FREE_MOVE_EVALS = 6;
constexpr int UNSEEN_RADIUS = 3;
constexpr double P_UNSEEN_ATTACK = 0.5, P_UNSEEN_CAP = 0.3;
constexpr double ROUTE_CONTEST_LOST = 0.4;
constexpr int ETA_RADIUS = 30;
```

```
constexpr int ROUTE_DFS_TURN_BUDGET = 24000;
constexpr int ROUTE_REPLAY_SLACK = 8;
```

`main.cpp` also gains `##include <tuple>`, the four rule helpers, `tiles_within` and `enemy_density_x1e4_`. Its header comment adds: *v21 (continued): the ceil(L/4) free-steps rule throughout (free_moves); moves of several free steps; routing with time-aware paths, arrival in turns, who gets there first, and a survival check where each route ends (route.hpp).*

11 Part II testing (new movement rule, against v21 Part I)

11.1 Routing scenarios (`sim/scen_route.py`)

`room`. A king of length 15 next to a fully visible 3×3 walled room with a one-tile door and 3 pearls inside (eating them leaves 18 segments in 9 tiles), plus 2 pearls in the open.

- Both versions eat the open pearls first.
- The Part I version then plans a route into the room and spends 4+ turns walking at it; its move-level safety stops it at the door.
- The new version rejects every route into the room and goes exploring elsewhere.

`contest`. A worker of length 6 (2 free steps) and an enemy head 2 tiles from pearl P1 by grid distance, but behind a kelp wall (30+ steps by path); pearl P2 is 1 step further for us and uncontested.

- The Part I version gives P1 up to the enemy and eats nothing in 4 turns.
- The new version takes P1 and then heads for P2.

11.2 Games against v21 Part I (both under the new rule)

Seed	Map	New as	New max	Old max	Result
21	20×36	A	177	90	new wins
21	20×36	B	174	98	new wins
22	18×25	A	–	–	new wins (old eliminated, round 86)
22	18×25	B	–	–	new wins (old eliminated, round 76)
20	56×53	A	296	97	new wins
20	56×53	B	293	142	new wins
30	44×61	A	220	89	new wins
30	44×61	B	259	100	new wins

8–0. These games were played before the one-line route-end offset fix (§11.5). Seed 30 re-run with the final build: new wins both again (max 290 vs 95 as A, 310 vs 136 as B). By round 500 the new build is down to 1–2 units (its workers have fed the king), against the old build's 9–11. On total length the old build was ahead in both games (638 vs 562, 836 vs 310), so under the current tiebreak (longest dragon first) the new build wins, but the totals are worth watching once the queen rule is implemented.

11.3 Metrics (seed 20, 56×53, 200 rounds)

	New	v21 Part I
Pearls eaten	8791	4611
Total length at round 200	1052	627
Segments lost to enemy trades	2382	1044
Our dragons of 16+ hit by enemy-started trades	46	9
Accidental self-deaths (segments)	294 (3896)	251 (2012)
Trapped deaths, length 10+	122	68

Free moves almost double the food. They also make our workers far longer at the unit cap: 40–140 where the old build's are under 50. Long workers are juicy targets and die whole when trapped. Over full games, self-deaths of dragons of 16+ ran about 2.5× the old build's (137–169 vs 62–67 per game on big maps).

11.4 Other checks

- The original scenario tests (`scen.py`) all give the same outcomes as before.
- Coil stress test (`stress_box.py`, 30 trials): no king self-deaths, for either build.
- Turn time (seed 20, 250 rounds, debug build): at most 6.1 ms, 1.4 ms on average (the Part I build: 2.7 ms, 0.9 ms).
- No crashes, timeouts or pay deaths in any game.

11.5 How Part II got here

1. The first version let the king make long free moves to the window's edge. The two king deaths above led to `unseen_risk` and the estimate-based soft reach.
2. `std::unordered_map<int, std::pair<int, Direction»::operator[]` does not compile (`Direction` has no default constructor), so `insert_or_assign` and `at` are used.
3. The route-end search first counted the other dragons' moves one turn ahead (offset `turns_for`(t) instead of `turns_for`$(t-1)$. That was slightly optimistic and is now fixed.

Part III
Landing safety and teammate memory

12 What was actually happening

The concern was that long snakes, able to take 20 free steps, move far past their window and so move blindly into teammates. They do not.

- Free moves are capped at `FREE_MOVE_MAX_STEPS` = 6 steps.
- Every step must be on a tile visible this turn, and `simulate_path` checks it against the board as it is.
- Dragons act one at a time, so nothing moves while we move.

What *is* blind is the landing. A multi-step move can end at the window's edge. Before our next turn, whatever is just outside the window (heads we cannot see, bodies we have forgotten) can close in, and the survival search cannot see it either. Bodies were also forgotten after 2 rounds, however long they were.

Measured (`probe_freemove.py`, seed 20, 200 rounds): how often a dragon was boxed in (no legal move) on the turn right after a move the survival search had called safe.

Length	After a multi-step move	After a single step	Multi-step, after Part III
10–29	28 / 2379 = 1.18%	1 / 959 = 0.10%	14 / 2536 = 0.55%
30+	4 / 803 = 0.50%	1 / 807 = 0.12%	4 / 897 = 0.45%
under 10	6 / 1673 = 0.36%	18 / 4561 = 0.39%	10 / 1592 = 0.63%

The walls around those heads were allies' bodies on 569 sides, our own body on 451, kelp on 123, ally heads on 81 and enemies on 44. Direct collisions with allies were rare (about 5 per 200 rounds).

13 Body memory

- `CellMemory::busy_until`: the round from which a body seen on the tile should be gone. It is set to $now + \texttt{OCCUPIED_MEMORY} + 1$ when the body is seen, which is the old 2-round memory.
- `remember_bodies()`, after `observe_heads`, refines it. Segment i of a visible head's body lasts $\lceil (len - i)/pace \rceil + 1$ rounds, at most $\texttt{BODY_MEMORY_MAX} = 30$, where:
  - *len* is `known_length` (for an ally, its last status report within 10 rounds, if longer);
  - *pace* is half the dragon's free steps for an ally (ours take several steps on most turns), and 1 for anyone else.
- A visible segment of an ally whose head we cannot see, but whose length we know, lasts half that length at its pace, clamped to 3–$\texttt{UNPLACED_BODY_MEMORY_MAX} = 8$ rounds.
- `body_remembered(m)` and `body_rounds_left(c)` replace the old fixed 2-round test. They are used by `remembered_occupied`, `blocks_pearl_path`, the time-aware space search and blind portal exits.
- The survival search frees a remembered tile after its rounds left (was a fixed 3); routing's `body_free_at_` does the same.
- The crowd count around route pearls still uses recent sightings only.

A first version assumed every body moves one step a turn. Measured over three maps, that was *worse* than having no long memory: our allies move faster, so it kept tiles closed too long and made the survival search pessimistic. The paced version measured slightly better than no memory.

14 Where allies can be before our next move

In `prepare_survival`, the step-1 head reach now uses what we know:

- **Allies**: $\min(\texttt{free_moves}(\texttt{known_length}), \texttt{FREE_MOVE_MAX_STEPS})$, since our allies never move further. **Enemies**: `free_moves(est)`.
- **Traced bodies** of visible heads use `known_length`.
- **Allies out of sight** that sent a status packet this round or the last: every tile within their reach of the reported head is blocked for our first step. The reach is doubled when the packet is from last round and the ally moves before us, since it has moved once since. The reported tile itself holds its body for $\min(30, len - 1)$ moves.
- `tiles_within` gained `any_first` (direction unknown) and now uses a set.

15 Landing next to tiles we cannot see

`unseen_risk` is now charged for every dragon from `position_value`, in two parts. With n the tiles within $\texttt{UNSEEN_RADIUS} = 3$ steps of the end that we cannot see now:

- **Ram** (valuable dragons, as in Part II): $\min(0.3,\ n \cdot \texttt{P_UNSEEN_ATTACK} \cdot \textit{enemy density}) \times \texttt{death_cost()}$.
- **Box-in** (any dragon of $\texttt{UNSEEN_BOX_MIN_LENGTH} = 10+$): $\min(0.3,\ n \cdot \texttt{P_UNSEEN_BOX} \cdot (\textit{ally} + \textit{enemy density})) \times \texttt{trap_cost()}$, with $\texttt{P_UNSEEN_BOX} = 0.05$. `ally_density_x1e4_` is a running average like the enemy one.

Every single step has $n = 1$, so single steps are not affected relative to each other.

Tried and dropped: a hard rule that long dragons end a free move at most 2 tiles from where they started. It measured no better than the price above.

16 The late-game king keeps to its feeders' pace

From `GATHER_ROUND` (380), the king and growers take no free moves while their goal is EXPLORE (outside a fight). They still take them to eat along a route, to fight, and to escape. Without this, the `feeder_near` scenario failed: a king of length 15 roamed 3–4 steps a turn and its length-4 feeder, at 1 step a turn, never reached it.

17 Part III results

Six large maps, 200 rounds each, both builds against the Part I build (`ab_trap.py`, seeds 20, 30, 37, 23, 27, 50):

	Part II build	Part III build	Change
Trapped deaths, length 10+	581	497	−14%
Accidental self-death segments	18054	16669	−8%
Total length at round 200	5237	5334	+2%
Segments lost to enemy rams	14320	15664	+9%

More long snakes stay alive, and some of them are then rammed instead.

Head-to-head against the Part II build, full games on large maps:

- the first version of these changes: 2–2;
- the paced memory: 1–3, every game by a small margin;
- the final build, seed 20 both sides: 1–1.

This is about even: the gain shows up in long-snake survival, not yet in wins.

Also found: every "ran into another body" death of ours is a single step through a portal whose exit we cannot see, onto an ally's body (`probe_hitbody.py`). About 40% of blind crossings die, the same before and after this round. This risk predates v21 and is priced at `P_BLIND_PORTAL` = 0.3.

Checks: the scenario tests give the same outcomes as before, including `feeder_near`, where the king is fed again. The cleaned-up build was confirmed move-for-move identical to the tested one before the late-game change.

Part IV
Going back for remembered pearls

18 What was wrong

In a game, a long king knew of pearls behind it but never went back for them.

- **Range in steps.** Routes considered only pearls within `KEEPER_ROUTE_RADIUS` = 20 steps (whole route: 28). A king of length 80 takes 20 free steps a turn, so a remembered pearl 30–40 steps away around its own body was never a route. The king fell back to exploring, and nearby unexplored ground (cost 12) always looked closer than a pearl 30+ steps away.
- **Speed capped by the window.** Even with a route, a free move only crosses tiles visible this turn. Towards a target outside the window that is 3 tiles of progress a turn, whatever the free steps.

Test (`scen_route.py loopback`): a king of length 40 is made to walk 60 steps west past three pearls, leaving them 22–25 steps behind with its body between, and then plays on its own.

Build	What happens
Before	No route: it explores away south and never eats them (20 turns watched).
Route range only	Goes back at 3 tiles a turn and eats all three 8 turns after taking over.
Final	Goes back and rushes 8 steps at once when the ground is quiet: all three eaten in 6–7 turns.

In this test the rush waits a few turns because the measured averages start from crowded-game values (see below).

19 Route range in turns

`route_radius()` and `route_horizon()` replace the fixed values: the old ones, or this many turns of our free steps, whichever is further:

- keepers: 4 turns (radius) and 6 turns (horizon);
- workers: 3 and 4 turns;
- capped at `ROUTE_MAX_RADIUS` = 100 and `ROUTE_MAX_HORIZON` = 140 steps.

A king of 40 now plans 40 steps ahead (60 for the whole route); one of 80, 80 and 120. Our time-aware map (`route_head_dist_`) is searched out to the same radius. Dragons with up to 5 free steps are unchanged.

20 Rush

`add_route_rush` runs when the goal is EAT. It follows the checked route path (`route_path_`) for all our free steps, never through a portal.

- **Moves weighed**: all of it, stopping on each route pearl, and the part we can see. A fully visible move of up to 6 steps is skipped, since it is already among the free moves.
- **Remembered tiles**: with `remembered_ok`, `simulate_path` also accepts a step onto a tile we cannot see now, if we have seen it, know the edge into it, and remember no body on it. Those tiles are returned as *blind*, and a pearl remembered on one counts as eaten (its tail staying put is the cautious case for self-collision).
- **Scoring**: `evaluate_path`, plus remembered pearls on blind tiles at their freshness, minus the chance of hitting something times `death_cost()`.
- **Safety**: a rush with more than `RUSH_MAX_RISK` = 8% (valuable dragons: 2%) chance of a hit is not safe. Rushes are exempt from the known-space penalty, since they price unseen tiles themselves.
- The chosen move logs as `RUSH`.

Chance a blind tile is taken: a tile last seen k rounds ago risks

$$\min(k \cdot \textit{fill rate},\ \textit{occupancy},\ \texttt{P_BLIND_TILE_CAP} = 0.5),$$

and a tile our own body is on now risks 0. Nobody else can be there, and nothing else moves while we do, so following our own body round is safe however far it goes. Both rates are measured in `remember_vision` as running averages:

- the fill rate: of the tiles we saw free last round and see again, the share now taken by another dragon;
- occupancy: the share of tiles in view holding another dragon.

They start at 3% and 20%. In a real game around the king they ran 3%/round and 23% in the midgame, and 1.5% and 15% late. So in crowds a tile seen 7+ rounds ago is a 20%+ risk and rushes are rare; in quiet ground or along our own body they go.

21 Bodies running out of sight

`predict_hidden_segments()` runs after `remember_bodies`.

- A visible segment's facing points to the next segment towards its head. If that tile is out of sight, it holds the same dragon.
- If no visible segment follows a segment towards its tail, and only one tile out of sight could, that tile almost surely does.
- Both tiles are remembered as taken for as long as the segment next to them, so rushes, the survival search and routing treat them as bodies. A remembered body no longer needs the tile to have been seen.

A rush in testing hit an ally's segment 3 just outside the window, where its body ran out of sight. Rush collisions went from 5 to 2 in 300 rounds with this.

22 The 12-bit id bug (predates v21)

Status packets, beacons and king records carry dragon ids in 12 bits. Both teams share one id counter, and with free moves (more eating, more splitting) it passed 4800 in a game. The king, id 4821, was reported as 725. Feeders looked for a head with id 725, never found it, never fed it, and wandered at lengths 150–215 until they died in their own coils. The team was wiped out in 2 of 6 games.

The fix: `pid(id) = id & 4095` and `my_id()`. Every id we store (`SeenHead`, signals, the king record, the ally length cache) and every comparison between a stored id and a real one uses that form; comparisons between two real ids (a visible part against our own id) are unchanged. Two live dragons whose ids differ by exactly 4096 would be confused, which is rare.

23 Part IV results

Full games, large maps, both sides:

New build	Against	Result	Final max length (avg)
Range, rush, hidden bodies (no id fix)	Part III build	4–2, both losses team wipeouts (id bug)	–
Final (with the id fix)	Part III build	**4–0** (seeds 30, 37)	413 vs 245
Final	Id fix only	**5–1** (seeds 20, 30, 37)	348 vs 278

So both parts pay: the id fix keeps the king fed, and the longer routes and rushes add about a quarter to its final length on top.

Over six maps of 200 rounds (before kings matter) the variants differ by about ±3% in total length, within run-to-run noise. Rushes eat about 3% more pearls there.

Checks: the scenario tests are unchanged (`room`, `contest`, the feeder tests) and `loopback` passes. The cleaned-up build plays move-for-move like the tested one.

Part V
The king's own moves

24 Deep self-safety

Until now a move was "proven safe" when the survival search found $\min(L+2, 20)$ moves. For a king of 200 that proves almost nothing: its newest segments stay put for up to 200 moves, so it could walk into a ring of its own recent body holding 25 tiles and be called safe.

- `deep_dragon()`: the king (also before growers start at round 300) and growers.
- For them, `survival_from(..., deep)` searches to $\min(L + 2, \texttt{KING_DFS_MAX_DEPTH} = 300)$. The budgets are `KING_DFS_NODE_BUDGET` = 20000 per search and `KING_DFS_TURN_BUDGET` = 250000 per turn (`KING_ROUTE_DFS_TURN_BUDGET` = 80000 for route ends, which are deep for them too).
- In deep searches, another dragon's body that we cannot place (head out of sight) is assumed gone after `DEEP_UNPLACED_BODY` = 40 moves rather than never, or a long search would fail on it.
- `survival_from` now takes a budget pool (0 moves, 1 route ends, 2 plans) and a per-search node cap.

Test (`scen_route.py ownpocket`): a king of 60 walks (scripted, so it knows its body) east and then round a full ring; at hand-over its newest 23 segments enclose 25 tiles holding pearls, and its head is at the ring's one gap.

Build	Stepping into the ring
Part IV build	*safe* (20 of 20 moves proven); kept out only by its score.
Final	*unsafe*: only 23 of the 62 moves needed exist; heavy penalty.

Both builds wait outside and eat the pearls once the ring has moved on, with 3 pearls inside or 7.

25 A body we do not know

A dragon born long from a split (an heir) starts as a new process: no map memory, and of its body only what it can see. On average heirs knew 79% of their body (`probe_bodyk.py`). The rest is attached where the known body ends (`own_trail_.back()`). In deep searches, while some of it is still there, a tile we have never seen within $\min(\textit{unknown}, \texttt{UNKNOWN_BODY_RADIUS} = 10)$ steps of that end counts as possibly our own body. (Multi-step moves never caused a body-model reset; `probe_bodyk.py` counted none.)

26 The king's plan

`plan_king_moves` runs after all move candidates exist, for deep dragons with 2+ free steps.

- For each of the `PLAN_CANDIDATES` = 8 best moves (safe ones first), the turns after it are searched. For turns 2..`PLAN_TURNS` = 3, a state's options are a shortest path to every tile within our free steps (at most `PLAN_MAX_STEPS` = 24, no portals). The search is time-aware against our own moving body. The `PLAN_BRANCH` = 4 best are kept, by food on the way and progress along the route (`plan_options`).
- Each option is played out exactly by `plan_extend`. Every step is checked against our body as the step before left it, tail included. A pearl, seen or remembered, eaten keeps the tail.

- Other bodies are timed cautiously (`plan_free_at_`): a tile another head can reach may hold its body for its length, not just for our next move. A plan should not count on gaps another dragon can fill.
- **A plan counts only if it is proven safe**: from where it ends, the deep survival search (budget pool 2, `PLAN_DFS_TURN_BUDGET` = 150000) must find a way on for our whole length.
- The best safe plan's *food* in its later turns, discounted `PLAN_DISCOUNT` = 0.85 a turn, is added to the move (`Candidate::plan_value`, ≥ 0; -1 means none was safe). Progress is left to the move's own score.

Two dead ends on the way

- **Counting progress in the plan made the king hold back.** Later turns "made" 10 steps of progress each through tiles the real move would price as risky, so a single step now left more for the plan to collect. In `loopback` the king crept one step a turn (pearls eaten at turn 78, not 66). Food only fixed it.
- **Cautious timing in every deep search** (not just in plans) made kings doubt every move in crowds. It lost 3 of 3 games against the Part IV build and was dropped. Used only inside plans it went 4–2; without it, plans went 1–5.

27 How kings actually get trapped

- **Self-deaths of long dragons by role** (`probe_selfdeath.py`): about 9 per game are kings or growers. Most are dragons born from a split one or two turns earlier, already boxed in.
- **Escape splits** (`probe_kingtrap.py`, `probe_ksplit.py`): the king escape-splits on about **1 in 10** of its turns (56–89 times a game, in both builds). Each split costs the head and 2 segments and restarts the king as a new process that knows little of its body.
- **Where they come from**: about two thirds are "blind" splits by young heirs that do not know their body. These are chains: the head alternates between the two ends of a body boxed in at both. The rest are kings that knew their body and were boxed in mostly by *other* dragons, often right after a multi-step move (`probe_kroom.py`).
- The deep check, the unknown-body caution and the plan did not change that rate.

28 Part V results

- **Final build vs the Part IV build**, six full games on large maps, both sides: **4–2**. Final king length averaged 346 vs 338, which is within the large game-to-game swings.
- Scenario tests unchanged; `loopback` still eats at turns 66–67; `ownpocket` as above.
- Coil stress test: no king self-deaths.
- Turn time: at most 36 ms (the king's plans and deep searches), 1.7 ms on average.

29 Known limitations and not yet done

- **Free-steps assumption**: per turn, from the length at the start of the turn. If the real engine counts differently, `free_moves` and `engine.py` are the two places to change.
- **Queens / win condition**: not implemented.
- **Teammate memory is partial**: we remember what we have seen and what status packets tell us (head position and length only). Where an unseen ally's body lies is guessed.
- **Blind portal crossings** die about 40% of the time in crowded games.

- **Rushes** use only free steps. Paying segments to rush to a remembered pearl is not considered: the pearl may be gone.
- The fill rate and occupancy start at crowded-game values, so a newborn long dragon does not rush for its first 30 turns.
- **The king's escape-split chains** (§27): about 1 split in 10 king turns, mostly by heirs that do not know their body. The most valuable next step for the king.
- **Crowding at the unit cap** is now the biggest leak. Workers can no longer split there and grow very long with free moves; they then die whole when trapped or rammed. This is the most valuable next step (for example, feeding long workers to the king or capping worker length).
- The fight search still gives the enemy single-step replies plus a sprint into our head; free repositioning of 2+ steps is not played out.
- Free moves avoid portals and stay inside our window.
- `unseen_risk` counts unseen tiles; it does not use what we saw there last turn.
- Route ends: the other dragons' future moves are unknown. A body whose head we cannot see stays a wall forever.
- Edges never seen are assumed open (as before), so a route into unseen ground can look better than it is until we see it.
- The survival search covers our own single steps only; other dragons are bodies that drain away plus a head reach.
- Pearls in the survival search count only if seen this round.
- Planned but not started: panic fallbacks.

Part VI
v21 → v22: the judge's points budget

30 What the judge measures

From the Timeouts page and the organisers' public repository (`unswcpmsoc/unswbc`, `pythoncli/unswbc/metering.py` and `sandbox.py`, which reproduce the judge):

- Bots run as WebAssembly (C++: `-O2 -msimd128`, libc++). Each instruction has a fixed price in *points*: most cost 1, loads and stores 2, division 3, calls 4–6, memory growth 50, bulk copies 10 plus 1 per 8 bytes.
- **100 million points per dragon per turn**, no carry-over. The turn runs from reading the round to the flush in `end_turn()`; work after that counts for the next turn. A process's first turn includes its own setup.
- A write to stdout costs 2.5M plus 4,000 a byte; a read 6 a byte. The helper buffers the whole turn into one write (one 2.5M charge a turn); `output_log` lines cost only their bytes.
- **The clock counts points**: `clock_time_get` returns points spent (1 ns = 1 point). So `std::chrono::steady_clock` inside the bot is an exact meter of its own spending.
- A turn over budget is stopped and delivers nothing. The action stays at its default, suicide, and the engine records `DragonDeathReason::NoValidAction`: the "no valid action" deaths.

The engine (`engine/src/actions.cc`) also uses `NoValidAction` for a paid step at length 2, a split that would leave fewer than 2 segments on either side, and a split at the unit cap. v21 already

avoids all three: paid steps are capped at $L - 2$ (`max_turn_steps`), and every split goes through `can_split`, whose unit count is current at our turn. So the deaths were the points budget.

31 Measuring it

`unswbc run --sandbox` measures exactly, but the toolkit ships through PyPI, which this workspace cannot reach. So the judge's build was reproduced:

- wasi-libc, libc++ and compiler-rt built from source for `wasm32-wasi` with clang 18 (`wasm/setup_toolchain.sh`); the bot built with `-O2 -msimd128 -mbulk-memory` (`wasm/build_bot.sh`). Exceptions are off; the judge's clang 20 with wasm exceptions may cost a little more.
- The organisers' own `metering.py` adds the point counter (`wasm/meter/instrument.py`).
- `wasm/meter/runner.mjs` runs it under Node's WASI the judge's way: the clock returns points, writes and reads are charged, and each turn's points are logged.
- The bot is deterministic, so a game is recorded natively (each dragon's input, `rec_bot.sh`) and every dragon is replayed through the meter (`replay.py`, `analyze.py`). A `BOT_PROFILE` build adds the points of each phase (`prof_an.py`).

32 v21's turns

One recorded 500-round game (seed 20, 56×53): our dragons of length 30+ and a sample of shorter ones, 7,410 turns, in millions of points.

Length	Turns	Median	p90	p99	Max	Over 100M
2–9	1715	20.3	33.3	81.9	150.6	13
10–29	1700	28.6	53.0	95.5	116.3	8
30–59	2702	40.8	102.7	173.3	299.2	292
60–99	933	39.6	200.4	351.5	934.7	252
100–149	210	27.7	312.0	536.4	840.8	75
150–199	77	30.4	575.4	1217.5	1217.5	31
200+	73	285.6	578.8	850.8	850.8	61

Kings averaged **164M a turn** (max 1.2B). Of that:

- 84M (51%) was the king's plan, almost all in `plan_options`: for every tile within 24 steps it copied the path and looked for the tile along the whole body (up to 300 segments) and the route.
- 50M (31%) was routing: 14 whole-map searches for the pearls' fields and the route replays.
- A survival-search position cost about 2,800 points, mostly `step_from` (edge copies, portal lookups in a `std::map`).

Short dragons sat at 15–20M a turn, most of it map-wide setup.

33 The turn's clock and deadlines

- `clock_now()` is `steady_clock` in nanoseconds: points in the judge. Native builds (local tests) multiply by `NATIVE_POINTS_PER_NS` = 24, measured against the metered build (p10–p90: 17–32).
- `main` starts the turn's clock before reading the round (the first turn: at program start).
- Deadlines, counted from the turn's start:

  - the route's checks stop at `ROUTE_STOP` = 40M;
  - the moves' survival searches at `MOVES_STOP` = 66M (first 64M);
  - the king's plans at `PLAN_STOP` = 91M (first 82M);
  - anything else at `SEARCH_STOP` = 92M (first 86M).

  The results below were measured with the first values. §36 raised them to a 95M peak. The phases are absolute, so a turn whose setup is expensive just leaves less for the later phases.

- One survival search may spend at most a quarter of what its phase has left (at least `SEARCH_MIN_POINTS` = 2M). A search deep in a tight spot can no longer use up the phase before the escape sprints are searched.
- Where the clock is read:
  - `survival_from`: before starting, and every 128 positions;
  - `plan_king_moves`: before each candidate and each state's options;
  - `plan_route`: after each pearl's field (fewer pearls are kept when time is short) and before each replay.
- **A search cut by time is treated like one cut by its node budget**:
  - a candidate's survival falls back to the room estimates;
  - a route end falls back to the free-neighbour check;
  - a plan that cannot be proven adds nothing.

  The node budgets stay; the deadlines only make sure the turn is never over.

34 Cheaper searches

- **Neighbour table.** `step_from` for every tile and direction lives in `nbr_` (the tile stepped onto, `NBR_WALL`, or `NBR_BLIND` for a portal whose far end we do not know), plus `nbr_portal_`. It is kept from turn to turn and refreshed once a turn, after the sonar and the vision are learned.
  - Only tiles whose edges changed are rebuilt, with their four neighbours: a step reads the far side's edge when its own is unknown.
  - A newly learned portal end rebuilds everything.
  - `step_from` itself reads the table, so every caller got faster. The survival search, the plan, the route fields, `timed_bfs`, `head_eta_bfs` and `replay_leg` index it directly.
  - `NBR_CHECK` compares it against the uncached `step_from` after the refresh and again mid-turn: no mismatch in two games. A deliberately broken refresh gives 133.
- `plan_options` **in one pass.**
  - Our body's tiles go in a stamped table (O(1) per tile instead of a walk along the path).
  - Food and the route pearls eaten (a bitmask) are carried along the search tree.
  - The route's remaining length is cached per set of pearls eaten: only its first leg depends on the tile.
  - Only the best `PLAN_BRANCH` options are kept, without sorting all of them, and nothing is allocated per call.
  - One behaviour change: pearls on tiles the plan already ate no longer count as food again.
- **Hash maps out of hot loops**: `replay_leg`'s parents and `head_eta_bfs`'s segment indices are stamped arrays, and `route_bfs` uses a vector queue.

- **The survival search's body is a ring buffer** instead of a `std::deque`. `SURV_CHECK` runs both on every search: no mismatch; a broken `back()` gives 22. It saved only about 4% a position.

A survival-search position now costs about 1,100 points (was 2,800). For kings of length 60+, `plan_options` fell from 103M to 5–12M a turn and the route fields from 43M to 8–12M.

35 Part VI results

Points (the same game and dragons as above, release build):

Length	Turns	Median	p90	p99	Max	Over 100M
2–9	1715	14.4	18.5	31.4	47.5	0
10–29	1700	17.8	24.1	31.1	36.6	0
30–59	2702	21.5	33.4	43.0	83.5	0
60–99	933	22.0	44.9	64.7	85.4	0
100–149	210	21.3	60.6	85.5	85.8	0
150–199	77	23.1	85.3	86.0	86.0	0
200+	73	59.4	85.2	85.6	85.6	0

- Kings now average 36.5M a turn: route 10.5M, plan 9.7M, free moves 4.5M, objective map 4.4M.
- Over four recorded games (15,225 turns of the dragons that reached length 40), the worst turn is **86.6M**. With the earlier 82M stop, every turn of every dragon in two v22 games (52,359 turns) stayed at or under 82.7M.
- Peak memory is 2.1 MB of the 48 MB allowed.
- The work after the last deadline (choosing, the output) is under 1M. The 13M margin is for the judge's build, which is not exactly ours.

How much lookahead the budget costs. On recorded inputs, v22 with deadlines off (`BOT_UNLIMITED`) chooses the same move as v21 on 2,244 of 2,258 turns. The 14 differences are all in one 200+ king and come from the plan's new tie-breaks. With deadlines on, v22's move differs from the unlimited one on:

- 0 of 2,047 turns at length < 100;
- 1 of 88 at 100–149;
- 7 of 83 at 150+.

At length 200+, the plan is cut on 8 of 73 turns and a single search's share on 14. The route and the moves are rarely cut (3 and 1).

Games. `engine.py --tle-ms` kills a dragon whose turn takes longer than that many wall-clock milliseconds, as the judge would. 4.5 ms is about 100M points at 24 points/ns. Each game ran on its own (`ab_tle.sh`).

Games	Result	Longest at the end (v22 / other)	Limit deaths per game
v22 vs v21, limit on: seeds 20, 6, 27, 30, 20 (v22 second in the last two)	**5–0**	81–139 / 0 (wiped out)	v22: 1–2. v21: 105–278, of them kings 49–99
v22 vs v22, limit on: 2 games	–	–	0
v22 vs v21, *no* limit (v21 unmetered): 3 games	0–3	170–194 / 392–456	–
v22 with deadlines off vs v21, no limit: 2 games	1–1	251, 354 / 271, 299	–

Two more limited games, recorded for the replays above (seeds 6 and 27), also went to v22 with v21 wiped out. **Correction (see §41):** native v22 in these simulator games ran without its searches, because of a bug in its native turn clock. So the "no limit" rows compare v21 with a v22 that had no lookahead, and do not measure what the budget costs. The metered replays, the move comparisons (run from recorded files) and your sandbox runs were not affected.

Under the limit, v21 is wiped out in every game. Its long dragons and kings die to the meter, which is what was seen in real matches. Without a limit, v21's unmetered lookahead (up to 1.2B points on a long king's turn) does buy strength. v22 gets as much of it as 100M allows.

v22's own limit deaths (7 in the 5 games against v21, none in self-play) took 4.9–5.8 ms, twice about 30 ms, and one was not timed. They are wall-clock noise of the local machine: native v22 stops its searches at about 3.6 ms, and the metered replays show no turn over 86.6M.

36 Checked in the sandbox, and a 95M peak

The official sandbox (`unswbc run -sandbox`, v21 as team A, v22 as team B):

Team	p50	p99	Mean	Max
v21	26.3M	52.1M	27.0M	99.7M
v22	17.1M	25.8M	17.1M	85.3M

- v22 won easily: its longest dragon reached 72, v21's 34.
- Seven v21 dragons died to the limit, all of them kings. A turn over budget shows as just under 100M because the sandbox stops it there.
- v22's 85.3M peak agrees with the metered replays here (86.6M), so the rebuilt toolchain prices the bot like the judge.

The peak is the plan's deadline plus a fixed tail. The metered replays were re-paired line by line: on replays from a file, the runner's round labels can run ahead, because the bot's input buffer reads ahead.

- The plan overshoots its stop by at most 0.8M (it ended at 82.81M at most against an 82M stop).
- Everything after it costs at most 4.1M on the 73 turns whose plan ended past 75M. That is the choosing, `send_status`, and the 2.5M write plus 4,000 a byte. The profiling build's own line is included; the release build is about 0.7M less.

So `PLAN_STOP` = 91M with `SEARCH_STOP` = 92M. `SEARCH_STOP` is kept just above the plan's stop, because a search after the plan (a split escape, say) could otherwise run to it and then pay the tail.

Measured over the same four recorded games (15,225 turns of the dragons that reached length 40):

- p50 18.2M, p99 66.8M, max **95.05M**;
- one turn over 95M, none over 96M.

This leaves 5M for anything the judge prices differently from the rebuilt toolchain. Its own run measured 85.3M where the replays gave 86.6M.

37 Tools added

- `wasm/setup_toolchain.sh`: builds the wasm32 sysroot (wasi-libc, libc++, compiler-rt builtins) with the system clang.
- `wasm/build_bot.sh SRC OUT.wasm [flags]`: the judge-like build.
- `wasm/meter/instrument.py` (with the organisers' `metering.py`): adds the point counter.
- `wasm/meter/runner.mjs`: the metered runner (`METER_BUDGET`, `METER_LOG`).
- `wasm/rec_bot.sh`, `wasm/replay.py`, `wasm/analyze.py`, `wasm/prof_an.py`: record, replay and summarise.
- `sim/engine.py --tle-ms`; `sim/ab_tle.sh`.
- Build flags:
  - `BOT_PROFILE`: a `PROF` line of points per phase, sub-timers, nodes and the deadlines hit;
  - `NBR_CHECK` and `SURV_CHECK`: the equivalence checks above;
  - `BOT_UNLIMITED`: no deadlines, for tests only.

With the official toolkit installed, `unswbc run <map> a b --sandbox` gives the judge's own numbers. It is worth running once on a long game to confirm the margin.

38 v22: known limitations and next steps

- **Setup costs every dragon 12–15M a turn**: objective map, survival preparation, ETAs and parsing. It is unguarded but bounded by the map size; trimming it would give every phase more.
- Long kings (200+) still have their plan cut on about one turn in nine (8 of 73). The next savings are free moves (about 15M on those turns), the route fields (12M) and the plan's option search.
- The deadlines are fixed shares of one turn. Search depth could be traded between phases by role. The budget resets every turn; nothing can be saved for a hard turn.
- The native factor (24 points/ns) is a median. Native test games only approximate the judge; the metered replay is the reference.

Part VII
v22 (continued): the queen

39 The rule

From the engine (`engine/src/scoring.cc`), at the last round the ranking is:

1. the queen's length (dragon 0 or 1, the first dragon each team's map lists; a dead queen counts 0);

2. the longest dragon;
3. total length.

A team that is wiped out loses at once. A split keeps the parent's id with the head part, so a queen that never splits stays the queen. The official maps give each team 1 to 9 starting dragons (`arena` and `Colosseum`: 1; `default`: 4; `australia`: 9).

40 What the queen does (`queen.hpp`)

- **Who**: `is_queen()` is `controller_.get_id() < 2`, the raw id. Our packets' 12-bit ids wrap past 4096, so heads we see carry their own `SeenHead::queen` flag, and beacons carry a queen bit (bit 18).
- **No splits**: no worker split (`should_split`), escape split (`play_split_escape`) or pocket raid (`pocket_plan_allowed`). Trap pricing assumes she cannot split out (`trap_cost`).
- **Growing the swarm first** (revised; see §44): until round `QUEEN_SEED_END_ROUND` = 75 she plays as a founder and makes the usual small worker splits. The head part, and her id, stays with her. (First version: only when she started alone, until we were 4 or round 60.)
- **The cycle, from round 75 until** `QUEEN_CYCLE_END_ROUND` = 400: a Hamiltonian cycle of a $w \times h$ block around her head (row 0, a serpentine over the other columns, back up column 0; h even).
  - The block is the largest size from 16 × 16 down, at most max(144, $2(L +$ `QUEEN_CYCLE_SLACK`)) tiles. Every step of its cycle must be an ordinary step (no kelp, no portal), by the neighbour table.
  - Of the placements, the one with the fewest other bodies on it and the fewest tiles never seen wins.
  - She walks it with as many of her free steps as are *queen-safe*: the usual safety, plus no enemy, ally, reach or soft threat at the end, and her whole length proven to have a way on (or two exits if that search was cut). Paid steps are never used.
  - The route search and the king's plan are skipped while she cycles.
- **Her body on the cycle**: walking it is safe only while every tile ahead is free of her own body by the time she reaches it (`queen_body_fits`). A segment i places back is gone after $L - i$ of her steps, one more for each pearl she eats on the way (the known pearls ahead plus `QUEEN_GROWTH_MARGIN` = 2); at least `QUEEN_CYCLE_MIN_FREE` = 4 spare tiles are required. She only follows the cycle while this holds. A new block (kelp found, or within 12 of her length) must pass the same test, and is never smaller than a cycle that still fits.
- **Off the cycle** (its next tiles are not queen-safe, or no cycle fits her): she takes the queen-safe move that ends nearest the cycle. If there is none, she takes the usual best move, as a deep-searching long dragon that never splits. She looks for a cycle that fits every `QUEEN_RETRY_ROUNDS` = 3 rounds.
- **Crown, from** `QUEEN_CROWN_ROUND` = 380: she stands for king, and she beats everyone in `offer_king` and `elected_king`, so beacons carry her and feeders gather on her. Before 380 she does not stand, and her length reports are ignored for the king/vice roles, so the team still has a roaming king. From 400 she retains as the king, everyone else feeds her, and her cycle ends. If she dies, the normal election takes over.

41 A bug in the native turn clock

The turn's clock started before reading the round. That is right in the judge: waiting costs no points there, and work after `end_turn` is charged to the next turn.

Natively, though, the clock ran on while the bot waited for every other dragon to move. So in every native simulator game, every turn began past all its deadlines and **v22 ran no searches**: no survival search, no plan, no route checks.

- This was found when the queen's candidates all showed an unsearched survival result.
- Native builds now start the clock once the round is read (`main`). The WebAssembly build, the one the judge runs, is unchanged.
- Affected: the native games in Part VI (the 5–0 under the limit, the 0–3 without one, `loopback` being slower).
- Not affected: the metered replays (recorded files and the points clock), the move comparisons from files, and your sandbox runs.

42 The simulator's pearls are far too rich

On the official maps, pearl beds and respawn gaps are set tile by tile (`TILE x y minGap maxGap`). On `default.map` most tiles respawn every 1–3,849 rounds (about 1,900 on average), and some every 1–255 or 1–385. The simulator's random maps use 4–60 on almost every tile: 30–100× more pearls. That is why simulator dragons reach 300–450 while the official sandbox game ended with a longest dragon of 72.

- `engine.py -pearl-scale k` (and `probe_queen.py`'s 6th argument) stretch the simulator's gaps by k.
- The simulator now ranks by queen length, then longest, then total, and reports `A_queen`/`B_queen`.
- Everything tuned in the simulator so far was tuned for the rich economy. Loading the official `.map` files into the simulator is the proper fix.

43 Queen results (simulator, so indicative only)

- With the simulator's rich pearls she grew to 35–77 by round 45–50, and every cycle filled up; she died off the cycle. This is what led to the body test above.
- With gaps stretched 30×, she walks the cycle on most turns (48 of her first 50 in one game). Her deaths there were enemy rams at length 3–7 (rounds 51, 107, 189): an enemy queen's 2-step sprint, and workers' attack sprints, on turns when no move was queen-safe.
- Queen build against an identical build without the queen (`NO_QUEEN`), 4 founders, 30× gaps: 1–1. Side A was wiped out in both games whichever build it was (a quirk of the stretched simulator to look into). Its queen survived in the game it won.
- Metered (the judge's pricing), over 250 queen turns: 11.2M points on average, 37.1M at most.

These need checking on official maps in the sandbox, with one founder (`arena`) and several (`default`).

44 The queen grows the swarm until round 75

A non-splitting queen hurt our early game: on maps with few founders, one of them stopped growing the swarm. Now:

- **Until round 75** she is an ordinary founder (`queen_seeding()` is just `is_queen()` and the round). She makes worker splits (she keeps her head, so she stays the queen) and joins the centre rush on small maps.
- She never makes pocket raids, and makes escape splits only when she is certainly dead (§45).

- **From round 75**: her Hamiltonian cycle. From 380 feeders gather on her; from 400 she is the king.
- Gone: `queen_alone_` and `QUEEN_SEED_UNITS`.

Simulator (pearls 30× sparser; indicative only):

- 32×16, 3 founders, against the previous build: our swarm went 7 → 25 → 29 dragons (rounds 75, 100, 125), while the other side, with a non-splitting queen, fell to 1. The game still went to the other side on queen length: ours died and theirs lived, which shows how much the first tiebreak weighs.
- 60×46, 4 founders: she split until round 75, then walked her cycle on 86 of the next 100 turns, and was alive when the other team was wiped out.

45 When the queen is certainly dead, she saves the rest of her length

A split leaves the queen where she is, facing the same way: it cannot save her, only her length. So she splits only when nothing she can do saves her (`play_split_escape`). Every candidate move must be one of:

- a head trade;
- a move that leaves her head no free side (unless it eats a pearl, so the split can come next turn with one more segment);
- a move from which the survival search, run to completion, finds at most one more move.

Unlike other dragons, an enemy that could reach her does not count as certain death, since it may not come. When she is doomed:

- the rear $L - 2$ segments become a new dragon, if its end is not boxed in too;
- she, the 2 left behind, dies as she would have anyway;
- the "cheap way out" room estimates cannot veto it, because her test already searched every way on.

Pocket raids stay off for her, and trap pricing still treats a trap as her death.

Test (a test build that starts her cycle at round 1, in the simulator's rich economy, where she gets boxed in):

- at length 102 she split off 100 (the child had 120 tiles of room);
- at length 54 she split off 52 (72 tiles);
- each time her head had no room left, and she died the next turn.

Found while testing: seeding is risky for her. As a founder she splits at length 4 and keeps 2, and she joins fights like any worker. In three rich-economy games she died at length 2, boxed in among enemies, by rounds 26–39, in this build and the previous one alike.

Part VIII
v22 (continued): the centre rush on small maps

46 Which maps

Top teams head for the centre on Autarky (54×18), Devil, Portals, Prisoners Dilemma and Tower Defense (32×16), Stripes (24×12) and Weakhold (40×15), but not on the larger maps. Area does not separate them: Trophy (625) and Queen of Spades (875) are smaller than Autarky (972) and are not on the list. **The short side does**:

- the listed maps' short sides are 12–18;
- every other mid-size or large map's is 24 or more;
- the tiny maps (Arena 11×11, Small 16×8, Colosseum and Default Small 16×16) fall under the same rule.

So the rush is on when $\min(W, H) \leq$ `CENTRE_MAP_MAX_SHORT_SIDE` $= 20$. The bot knows W and H from its first turn.

Added: compact maps. The centre also pays on Trophy (25×25, whose centre beds are 3.8× richer than its edges). Its short side is 25, the same as Queen of Spades (25×35), which is not on the list. Area cannot separate them either: Autarky (972) is on the list and larger than Queen of Spades (875). So a second condition was added: the rush is also on when $\max(W, H) \leq$ `CENTRE_MAP_MAX_LONG_SIDE` $= 30$.

- **On**: Arena, Small, Colosseum, Default Small, Stripes, Devil, Portals, Prisoners Dilemma, Tower Defense, Weakhold, Trophy, Autarky.
- **Off**: Queen of Spades (both), Default (32×32), Maze, Stronghold, Trauma, Slithery Fight, Islands, Schooltime, and the 64×64 maps.

It is not about pearls. On Prisoners Dilemma, Weakhold and Autarky the centre's pearl beds give 2–9% of the outer rate; Tower Defense's give 15× it. On every listed map the teams start at opposite ends of the long axis, so the centre is the ground between them.

47 What it does

- For the first `CENTRE_RUSH_END_ROUND` $= 75$ rounds, every dragon gets `CENTRE_PULL` $= 300$ (thousandths of a segment) per step a move brings it closer to the centre. 450 was tried; 300 was kept.
- Exploring is worth 180 a step plus up to 120 for dispersal, so the centre wins over exploring. A pearl next to the dragon still wins over the centre.
- "Closer" is by path: a search from the centre over the neighbour table each turn of the rush (`prepare_centre`), with kelp and portals as we know them and unseen edges open. A straight-line pull would press dragons against walls.
- The pull stops within `CENTRE_RADIUS` steps of the centre (short side / 4, at least 2), so dragons do not pile onto one tile. It applies to every goal but fights and feeding (`goal_progress` now adds it to `goal_progress_base`).
- After round 75 it is off, so nobody gets stuck in the middle.
- The queen walks her cycle and does not rush (her moves are her own; see the queen part).

48 Simulator check (random maps, pearls stretched 30×: indicative only)

`probe_centre.py` gives the mean head distance to the centre (rush build vs current build):

- 32×16, 3 founders each: 4.0–5.7 vs 7.5–14.0 from round 25 to 75. Afterwards the rush build spreads out again (11.0 at round 100).
- 40×15, 4 founders each: 4.6–7.2 vs 11.8–16.6 from round 10 to 75.
- Both games, and the second with the sides swapped, left the rush team with far fewer dragons by round 75 (4–5 against 8–15). The likely reason is fewer pearls eaten on the way, so fewer splits. Whether this holds on the real maps, where top teams do rush, is for the sandbox to show; `CENTRE_PULL` is the knob to try lower (around 150, so pearls compete).